VENTURESIGNAL

AI STARTUP DUE DILIGENCE

Project Training Manual

A codebase-guided handbook for new developers, teammates, and project-review preparation.

Version 1.0 | 07 August 2026

Team: Not specified in the repository

What this manual is

This PDF was produced from the current repository. It explains what is actually implemented, including limitations and unused paths; it does not describe planned functionality as complete.

Table of Contents

Section page numbers are generated automatically when this PDF is built.

Table of Contents 2
1. Project Overview 4
Objectives . . . 4
Real-world use case . . . 4
2. Technology Stack 5
AI/OpenAI clarification . . . 5
3. Complete Architecture 6
Runtime architecture . . . 6
Architecture block explanation . . . 6
Training architecture . . . 6
4. Folder Structure 7
5. File-by-File Explanation 8
5.1 Backend and machine-learning files . . . 8
app.py . . . 8
predict.py . . . 8
preprocess.py . . . 8
train_model.py . . . 9
dataset_preprocessing.py . . . 9
column_mapping.py . . . 9
insight_generator.py . . . 10
chart_generator.py . . . 10
pdf_generator.py . . . 10
check_data.py . . . 11
5.2 Frontend and configuration files . . . 12
frontend/src/main.jsx . . . 12
frontend/src/App.jsx . . . 12
frontend/src/pages/Dashboard.jsx . . . 12
frontend/src/pages/About.jsx . . . 12
frontend/src/services/api.js . . . 13
frontend/src/components/UploadCard.jsx . . . 13
frontend/src/components/PredictionCard.jsx . . . 13
frontend/src/components/RiskGauge.jsx . . . 14
frontend/src/components/Statistics.jsx . . . 14
frontend/src/components/FeatureImportance.jsx . . . 14
frontend/src/components/Charts.jsx . . . 15
frontend/src/components/Hero.jsx, Navbar.jsx, Footer.jsx, Loading.jsx . . . 15

frontend/src/components/Sidebar.jsx . . . 15
frontend/src/index.css and App.css . . . 16
frontend/package.json, vite.config.js, eslint.config.js, index.html . . . 16
6. Project Workflow 17
Selected-company PDF workflow . . . 17
Step-by-step in plain English . . . 17
Single prediction workflow . . . 17
7. API Documentation 18
Request example . . . 18
8. Frontend Guide 19
User interaction map . . . 19
9. Backend Guide 19
10. AI Module 20
Decision process . . . 20
Limitations . . . 20
11. Report Generation 20
12. Data Flow Between Files 21
13. Libraries Used 21
14. Common Errors and Debugging 21
15. Future Scope 22
16. Team Handover Guide 22
First files to read . . . 22
Adding a feature . . . 22
Debug sequence . . . 22
17. Frequently Asked Viva Questions 23
18. Quick Revision 27
Five-page revision checklist . . . 27
Page A - Architecture . . . 27
Page B - ML . . . 27
Page C - Prediction . . . 27
Page D - Report . . . 27
Page E - Caution . . . 27
Final team checklist . . . 27

1. Project Overview

Project title: AI Startup Due Diligence Platform, presented in the web interface as VentureSignal.

Problem: startup screening can be inconsistent and slow. This project turns structured startup data into a historical outcome prediction and a selected-company due-diligence PDF.

Why it exists: to demonstrate an end-to-end ML product: data cleaning, supervised training, API inference, browser UI, spreadsheet handling, visual reporting, and downloadable output.

Objectives

- Train a multiclass startup-outcome classifier from historical records.
- Score an individual startup and show prediction, probability, confidence, and risk label.
- Read CSV/XLSX/XLS uploads, let the user select one record, and generate a report.
- Show model metadata, held-out metrics, and feature importance in the browser.

Real-world use case

An analyst uploads a portfolio spreadsheet, selects one company, and receives a PDF with the model result, available financial context, information gaps, risk areas, and a rule-based recommendation. The output is decision support only, not legal, financial, or investment advice.

> **Critical implementation fact**
>
> The codebase does not use OpenAI, an LLM, prompts, embeddings, or external AI APIs. Its AI module is a scikit-learn Random Forest and its report text is generated by deterministic Python rules.

2. Technology Stack

Technology	Why used	Where used / benefit
React 19 + React DOM	Builds a reusable, interactive browser UI.	frontend/src; component state and declarative rendering.
Vite	Fast development server and production bundling.	frontend package scripts and vite.config.js.
Python	Strong ecosystem for data, ML, APIs, and documents.	All root backend/ML/report modules.
Flask + Flask-CORS	Lightweight REST routes and browser cross-origin access.	app.py; exposes JSON/PDF endpoints.
pandas + openpyxl	Reads/manipulates CSV and Excel tabular data.	Training, upload ingestion, model input, and reports.
scikit-learn	Label encoders, split, Random Forest, and metrics.	preprocess.py, train_model.py, predict.py.
joblib	Persists Python ML artifacts.	Saves/loads model, feature order, and encoders.
ReportLab + Matplotlib	Builds a styled PDF and local chart images.	pdf_generator.py and chart_generator.py.
Axios, Recharts, Framer Motion, Lucide	HTTP, charts, animation, icons.	frontend service/components.

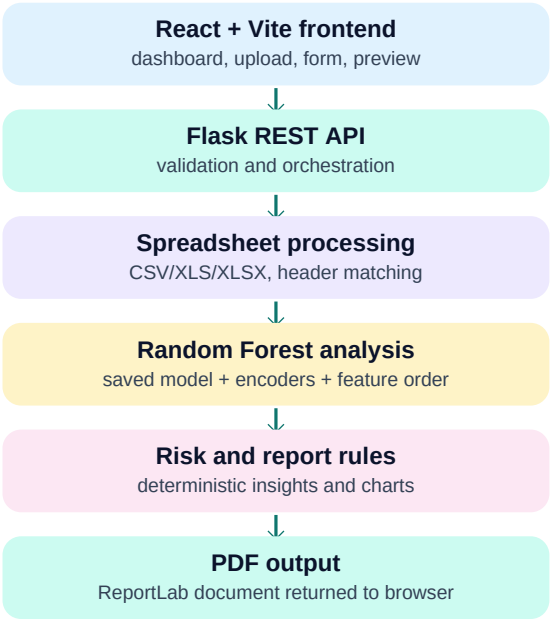
AI/OpenAI clarification

The requested category 'AI/OpenAI' is documented accurately: AI is used through the local Random Forest model. OpenAI is not present in requirements, imports, or runtime code.

3. Complete Architecture

The platform has two separate flows: offline model training and online browser/API/report execution.

Runtime architecture



Architecture block explanation

- Frontend: Dashboard.jsx gathers inputs, calls api.js, shows state and Blob PDF preview.
- Backend: app.py owns HTTP routes and converts request data to functions/HTTP responses.
- Excel processing: dataset_preprocessing.py reads uploads; column_mapping.py recognizes common business header names.
- AI analysis: predict.py loads joblib artifacts, aligns columns, encodes categoricals, and calls predict/predict_proba.
- Risk detection: get_risk_level() gives a simple UI label; insight_generator.py evaluates available business metrics.
- PDF generation: ReportLab assembles text/tables and embeds Matplotlib charts under output/pdf/.

Training architecture

data/CleanStartups.csv -> preprocess.py -> data/processed_startups.csv + model/label_encoders.pkl -> train_model.py -> risk_model.pkl, features.pkl, metrics.json, feature_importance.csv.

4. Folder Structure

The following is the application-relevant tree. Git/tool caches, Python bytecode, and temporary files are intentionally not source modules.

```
AI-Startup-Due-Diligence/
app.py | predict.py | preprocess.py | train_model.py
dataset_preprocessing.py | column_mapping.py | insight_generator.py
chart_generator.py | pdf_generator.py | check_data.py
requirements.txt | README.md
data/ CleanStartups.csv, processed_startups.csv
model/ risk_model.pkl, features.pkl, label_encoders.pkl, metrics.json, feature_importance.csv
output/pdf/ runtime-generated reports and chart PNGs
frontend/
package.json, package-lock.json, vite.config.js, eslint.config.js, index.html
public/ favicon.svg, icons.svg
src/
main.jsx, App.jsx, index.css, App.css, assets/
services/api.js
pages/Dashboard.jsx, About.jsx
components/ UploadCard.jsx, PredictionCard.jsx, RiskGauge.jsx, Statistics.jsx,
FeatureImportance.jsx, Charts.jsx, Hero.jsx, Navbar.jsx, Footer.jsx,
Loading.jsx, Sidebar.jsx
dist/ generated production frontend files
```

Location	What it contains	How it connects
data/	Raw and processed startup CSVs.	Feeds model training; CleanStartups supplies sample form data.
model/	Persisted ML artifacts.	Loaded by predict.py and exposed as metadata by app.py.
output/pdf/	Generated reports and temporary chart assets.	Written by pdf_generator.py and sent by Flask.
frontend/src/	React UI code, styles, API layer.	Calls Flask and renders results.
frontend/dist/	Built static files.	Generated by npm run build; never hand-edit.

5. File-by-File Explanation

This is the core teaching section. Each card identifies why the file exists, its trigger/caller, dependencies, key logic, data contract, failure behaviour, and a short viva script.

5.1 Backend and machine-learning files

app.py

Purpose. The Flask REST API entry point and application orchestrator.

When it runs / caller. Run by python app.py; create_app() registers routes. The frontend calls its endpoints through api.js.

Calls / dependencies. Calls read_startup_file/adapt_for_model/filter_company_data/get_company_list, predict_dataframe/load_prediction_artifacts, and generate_report.

Imports. Built-in: json, logging, pathlib, typing. Third-party: pandas, Flask, Flask-CORS, Werkzeug. Local: dataset_preprocessing, pdf_generator, predict.

Important logic. Routes: /, /health, /predict, /predict/csv, /reports/companies, /reports/generate, /model-info, /metrics. Helpers set file rules, row parsing, risk labels, and metadata loading.

Input -> output. JSON or multipart upload -> JSON response or PDF download

Error handling. PredictionError and BadRequest become JSON 400; 413 explains 10 MB upload limit; health returns 503 when artifacts cannot load.

> **30-second viva explanation**
>
> app.py is the backend gateway. It validates browser requests, calls the ML/report modules, and returns safe JSON or a downloadable PDF.

predict.py

Purpose. The reusable model inference pipeline.

When it runs / caller. Called by app.py health, prediction, batch, and report routes.

Calls / dependencies. Loads joblib artifacts from model/. Uses pandas and LabelEncoder.

Imports. Built-in: logging, pathlib, typing. Third-party: joblib, pandas, sklearn LabelEncoder.

Important logic. Validates artifact paths; loads model/features/encoders; reorders required columns; encodes categoricals; calls model.predict and predict_proba; decodes Fate classes.

Input -> output. Input DataFrame -> Copy of original DataFrame plus Prediction, Probability, Confidence

Error handling. Raises PredictionError for missing/corrupt artifacts, bad values, or absent Fate encoder. Unseen categories warn and fall back to 0.

> **30-second viva explanation**
>
> predict.py guarantees that inference uses the same model schema and label encoders as training before it makes a prediction.

preprocess.py

Purpose. Cleans the raw training CSV and creates training-time label encoders.

When it runs / caller. Called by train_model.py; can also run directly.

Calls / dependencies. Calls pandas CSV operations and joblib dump; exposes preprocess_data().

Imports. Built-in: logging, pathlib, typing. Third-party: joblib, pandas, sklearn LabelEncoder.

Important logic. Validates CSV path, drops accidental Unnamed index columns, removes duplicates, fills missing numerical values with median/zero and categorical values with mode/Unknown, encodes text columns, saves results.

Input -> output. Raw CleanStartups.csv -> Processed DataFrame, processed_startups.csv, label_encoders.pkl

Error handling. Uses PreprocessingError for bad path/CSV/empty data/output failures.

> **30-second viva explanation**
>
> preprocess.py makes historical raw data safe and numeric for model training while preserving encoders needed later at prediction time.

train_model.py

Purpose. Trains, evaluates, and persists the Random Forest classifier.

When it runs / caller. Run manually with python train_model.py after changing raw data.

Calls / dependencies. Calls preprocess_data() then scikit-learn split, fit, metric functions, and joblib/file saves.

Imports. Built-in: json, logging, pathlib, typing. Third-party: joblib, pandas, sklearn ensemble/metrics/model_selection. Local: preprocess.

Important logic. Uses stratified 80/20 split and random_state 42. build_model() returns 300-tree, max-depth-15 forest. evaluate_model() calculates accuracy, weighted precision/recall/F1, ROC-AUC, confusion matrix, report; saves artifacts.

Input -> output. Raw dataset via preprocessing -> Fitted model and model artifacts

Error handling. TrainingError wraps validation, split, training, metric, and persistence failures.

> **30-second viva explanation**
>
> train_model.py is the offline ML pipeline: it preprocesses data, trains a reproducible Random Forest, evaluates it on held-out data, and saves everything the API needs.

dataset_preprocessing.py

Purpose. Reads report uploads and adapts flexible spreadsheets to the fixed model schema.

When it runs / caller. Called by app.py upload/report routes.

Calls / dependencies. Calls column_mapping helpers and uses PredictionError from predict.py.

Imports. Built-in: pathlib, BinaryIO. Third-party: pandas. Local: column_mapping, predict.

Important logic. read_startup_file() supports CSV/XLSX/XLS. validate_report_dataframe() requires a usable company header. get_company_list() preserves original row identities. filter_company_data() returns exactly selected row. adapt_for_model() maps names/fills missing fields.

Input -> output. Upload stream or DataFrame -> Cleaned, selected, or model-aligned DataFrame

Error handling. Rejects unreadable/empty files, missing company field, invalid selection. Missing model fields are filled deliberately for prediction.

> **30-second viva explanation**
>
> This file separates user-friendly spreadsheet handling from strict ML feature requirements.

column_mapping.py

Purpose. Recognizes semantically similar spreadsheet headers.

When it runs / caller. Called by dataset_preprocessing, insight_generator, chart_generator, and pdf_generator.

Calls / dependencies. Only uses re and Iterable.

Imports. Built-in: re, typing. No third-party imports.

Important logic. ALIASES defines canonical business concepts. normalize_column() lowercases/removes punctuation. find_column() tries exact then partial alias matches. map_columns() builds all found mappings.

Input -> output. Column-name iterable + concept -> Actual column name or mapping dictionary

Error handling. Returns None when no match; callers handle absence.

> **30-second viva explanation**
>
> column_mapping.py lets reports understand headings like Company Name, Startup Name, Funding, or Total Funds without forcing one upload template.

insight_generator.py

Purpose. Creates evidence-based report text and risk rows without an LLM.

When it runs / caller. Called by pdf_generator.py for each selected company.

Calls / dependencies. Calls column-mapping functions and pandas numeric conversion.

Imports. Third-party: pandas. Local: column_mapping.

Important logic. number()/value() extract data; report_metric_values() keeps verified numeric business fields; report_context() creates recommendation/reasons/summary; risk_sections() returns Financial, Operational, Commercial, Execution rows.

Input -> output. One-row DataFrame + prediction dictionary -> Context dictionary, observations, and risk tuples

Error handling. Missing data leads to explicit disclosure-gap statements rather than invented values.

> **30-second viva explanation**
>
> insight_generator.py turns available uploaded metrics and model output into transparent, deterministic diligence language.

chart_generator.py

Purpose. Produces PNG financial charts for the PDF.

When it runs / caller. Called by generate_report().

Calls / dependencies. Calls find_column() and matplotlib/pandas.

Imports. Built-in: pathlib. Third-party: matplotlib, pandas. Local: column_mapping.

Important logic. For revenue, funding, employees, growth, burn, runway, valuation it finds a column, parses a numeric value, draws one horizontal bar, saves image, closes figure.

Input -> output. One-company DataFrame + output directory -> List of PNG paths

Error handling. Silently skips unavailable/non-numeric metrics; it does not fabricate a chart.

> **30-second viva explanation**
>
> chart_generator.py creates simple local visual evidence from metrics actually supplied in the selected row.

pdf_generator.py

Purpose. Creates the polished selected-company PDF report.

When it runs / caller. Called only by POST /reports/generate in app.py.

Calls / dependencies. Calls chart_generator, column_mapping, insight_generator; uses ReportLab and pandas.

Imports. Built-in: datetime, pathlib, html, re, time. Third-party: pandas, ReportLab. Local: chart_generator, column_mapping, insight_generator.

Important logic. Defines visual styles/header/footer, safe display/table helpers, profile/industry/SWOT logic, then generate_report() builds cover, summary, context, tables, features, charts, KPIs, risks, and appendix.

Input -> output. One selected row, prediction dict, saved metrics, output path -> Saved PDF path

Error handling. Requires exactly one record; optional feature-importance read failure is ignored; missing input metrics are disclosed.

> **30-second viva explanation**
>
> pdf_generator.py is the presentation layer: it turns the selected row and model result into a structured, locally generated due-diligence PDF.

check_data.py

Purpose. Small development inspection utility.

When it runs / caller. Only when a developer runs python check_data.py.

Calls / dependencies. Reads data/processed_startups.csv with pandas.

Imports. Third-party: pandas.

Important logic. Prints Fate value counts and DataFrame data types.

Input -> output. Processed CSV -> Terminal diagnostics

Error handling. No custom error handling; use only after preprocessing has produced the file.

> **30-second viva explanation**
>
> check_data.py is not part of the web app; it is a quick sanity check for processed labels and column types.

5.2 Frontend and configuration files

frontend/src/main.jsx

Purpose. Browser entry point. createRoot mounts App in StrictMode.

When it runs / caller. Vite loads it from index.html.

Calls / dependencies. See statement above; UI components generally call upward callbacks, while Dashboard calls api.js.

Imports. React createRoot, App, index.css.

Important logic. Input: #root element. Output: mounted React tree.

Input -> output. Props, browser events, or route -> Rendered UI or API action

Error handling. API errors are shown in Dashboard toast; rendering uses optional chaining/empty states.

> **30-second viva explanation**
>
> main.jsx is part of the React presentation layer. It keeps the interface modular while Dashboard and api.js coordinate data and backend communication.

frontend/src/App.jsx

Purpose. Router and app shell.

When it runs / caller. Mounted by main.jsx.

Calls / dependencies. See statement above; UI components generally call upward callbacks, while Dashboard calls api.js.

Imports. BrowserRouter/Routes, Navbar, Footer, Dashboard, About.

Important logic. Routes / to Dashboard and /about to About.

Input -> output. Props, browser events, or route -> Rendered UI or API action

Error handling. API errors are shown in Dashboard toast; rendering uses optional chaining/empty states.

> **30-second viva explanation**
>
> App.jsx is part of the React presentation layer. It keeps the interface modular while Dashboard and api.js coordinate data and backend communication.

frontend/src/pages/Dashboard.jsx

Purpose. Main stateful screen.

When it runs / caller. Rendered for / by App.jsx.

Calls / dependencies. See statement above; UI components generally call upward callbacks, while Dashboard calls api.js.

Imports. React hooks; components; functions from services/api.js.

Important logic. Loads health/model/metrics; controls form, prediction, upload company list, selected row, PDF Blob URL, toast, and loading states.

Input -> output. Props, browser events, or route -> Rendered UI or API action

Error handling. API errors are shown in Dashboard toast; rendering uses optional chaining/empty states.

> **30-second viva explanation**
>
> Dashboard.jsx is part of the React presentation layer. It keeps the interface modular while Dashboard and api.js coordinate data and backend communication.

frontend/src/pages/About.jsx

Purpose. Static product explanation page.

When it runs / caller. Rendered for /about.

Calls / dependencies. See statement above; UI components generally call upward callbacks, while Dashboard calls api.js.

Imports. Framer Motion only.

Important logic. Shows implementation claims: Random Forest, persisted artifacts, batch API concept.

Input -> output. Props, browser events, or route -> Rendered UI or API action

Error handling. API errors are shown in Dashboard toast; rendering uses optional chaining/empty states.

> **30-second viva explanation**
>
> About.jsx is part of the React presentation layer. It keeps the interface modular while Dashboard and api.js coordinate data and backend communication.

frontend/src/services/api.js

Purpose. Single Axios API boundary.

When it runs / caller. Imported by Dashboard.jsx.

Calls / dependencies. See statement above; UI components generally call upward callbacks, while Dashboard calls api.js.

Imports. Axios and VITE_API_BASE_URL.

Important logic. Exports getDashboardData, predictStartup, predictCsv, generateDueDiligenceReport, getReportCompanies; maps server errors to readable Error messages.

Input -> output. Props, browser events, or route -> Rendered UI or API action

Error handling. API errors are shown in Dashboard toast; rendering uses optional chaining/empty states.

> **30-second viva explanation**
>
> api.js is part of the React presentation layer. It keeps the interface modular while Dashboard and api.js coordinate data and backend communication.

frontend/src/components/UploadCard.jsx

Purpose. Spreadsheet file picker, drag/drop, selection UI, PDF action controls.

When it runs / caller. Rendered by Dashboard.

Calls / dependencies. See statement above; UI components generally call upward callbacks, while Dashboard calls api.js.

Imports. React useState and Lucide icons.

Important logic. Accepts callbacks/props. Client-side accepts csv/xlsx/xls; server performs authoritative validation.

Input -> output. Props, browser events, or route -> Rendered UI or API action

Error handling. API errors are shown in Dashboard toast; rendering uses optional chaining/empty states.

> **30-second viva explanation**
>
> UploadCard.jsx is part of the React presentation layer. It keeps the interface modular while Dashboard and api.js coordinate data and backend communication.

frontend/src/components/PredictionCard.jsx

Purpose. Displays latest single prediction.

When it runs / caller. Rendered by Dashboard.

Calls / dependencies. See statement above; UI components generally call upward callbacks, while Dashboard calls api.js.

Imports. Framer Motion, ArrowUpRight icon.

Important logic. Uses prediction, probability, confidence, risk_level; has an empty-state card.

Input -> output. Props, browser events, or route -> Rendered UI or API action

Error handling. API errors are shown in Dashboard toast; rendering uses optional chaining/empty states.

> **30-second viva explanation**
>
> PredictionCard.jsx is part of the React presentation layer. It keeps the interface modular while Dashboard and api.js coordinate data and backend communication.

frontend/src/components/RiskGauge.jsx

Purpose. Circular confidence visualization.

When it runs / caller. Rendered by Dashboard.

Calls / dependencies. See statement above; UI components generally call upward callbacks, while Dashboard calls api.js.

Imports. No external data library.

Important logic. Uses a conic-gradient and colour chosen from risk_level.

Input -> output. Props, browser events, or route -> Rendered UI or API action

Error handling. API errors are shown in Dashboard toast; rendering uses optional chaining/empty states.

> **30-second viva explanation**
>
> RiskGauge.jsx is part of the React presentation layer. It keeps the interface modular while Dashboard and api.js coordinate data and backend communication.

frontend/src/components/Statistics.jsx

Purpose. Four summary cards.

When it runs / caller. Rendered by Dashboard.

Calls / dependencies. See statement above; UI components generally call upward callbacks, while Dashboard calls api.js.

Imports. Lucide icons.

Important logic. Derives training samples, accuracy, features, ROC-AUC from dashboard API state.

Input -> output. Props, browser events, or route -> Rendered UI or API action

Error handling. API errors are shown in Dashboard toast; rendering uses optional chaining/empty states.

> **30-second viva explanation**
>
> Statistics.jsx is part of the React presentation layer. It keeps the interface modular while Dashboard and api.js coordinate data and backend communication.

frontend/src/components/FeatureImportance.jsx

Purpose. Horizontal Recharts importance chart.

When it runs / caller. Rendered by Dashboard.

Calls / dependencies. See statement above; UI components generally call upward callbacks, while Dashboard calls api.js.

Imports. Recharts.

Important logic. Receives top importance records from /model-info.

Input -> output. Props, browser events, or route -> Rendered UI or API action

Error handling. API errors are shown in Dashboard toast; rendering uses optional chaining/empty states.

> **30-second viva explanation**
>
> FeatureImportance.jsx is part of the React presentation layer. It keeps the interface modular while Dashboard and api.js coordinate data and backend communication.

frontend/src/components/Charts.jsx

Purpose. Model-metric and optional batch-distribution charts.

When it runs / caller. Rendered by Dashboard.

Calls / dependencies. See statement above; UI components generally call upward callbacks, while Dashboard calls api.js.

Imports. Recharts.

Important logic. Metric chart works. Batch chart needs batchResult, but Dashboard does not pass it in current implementation.

Input -> output. Props, browser events, or route -> Rendered UI or API action

Error handling. API errors are shown in Dashboard toast; rendering uses optional chaining/empty states.

> **30-second viva explanation**
>
> Charts.jsx is part of the React presentation layer. It keeps the interface modular while Dashboard and api.js coordinate data and backend communication.

frontend/src/components/Hero.jsx, Navbar.jsx, Footer.jsx, Loading.jsx

Purpose. Reusable visual/status/navigation elements.

When it runs / caller. Rendered by App or Dashboard.

Calls / dependencies. See statement above; UI components generally call upward callbacks, while Dashboard calls api.js.

Imports. Framer Motion, React Router, Lucide where needed.

Important logic. They do not call backend APIs.

Input -> output. Props, browser events, or route -> Rendered UI or API action

Error handling. API errors are shown in Dashboard toast; rendering uses optional chaining/empty states.

> **30-second viva explanation**
>
> Hero.jsx, Navbar.jsx, Footer.jsx, Loading.jsx is part of the React presentation layer. It keeps the interface modular while Dashboard and api.js coordinate data and backend communication.

frontend/src/components/Sidebar.jsx

Purpose. Placeholder navigation component.

When it runs / caller. Rendered by Dashboard.

Calls / dependencies. See statement above; UI components generally call upward callbacks, while Dashboard calls api.js.

Imports. Lucide.

Important logic. Currently display:none, so it has no visible user role.

Input -> output. Props, browser events, or route -> Rendered UI or API action

Error handling. API errors are shown in Dashboard toast; rendering uses optional chaining/empty states.

30-second viva explanation

Sidebar.jsx is part of the React presentation layer. It keeps the interface modular while Dashboard and api.js coordinate data and backend communication.

frontend/src/index.css and App.css

Purpose. Global, layout, glassmorphism, responsive styles.

When it runs / caller. Imported from main.jsx/App.

Calls / dependencies. See statement above; UI components generally call upward callbacks, while Dashboard calls api.js.

Imports. CSS only.

Important logic. No business logic.

Input -> output. Props, browser events, or route -> Rendered UI or API action

Error handling. API errors are shown in Dashboard toast; rendering uses optional chaining/empty states.

30-second viva explanation

index.css and App.css is part of the React presentation layer. It keeps the interface modular while Dashboard and api.js coordinate data and backend communication.

frontend/package.json, vite.config.js, eslint.config.js, index.html

Purpose. Frontend dependency/build/lint/HTML configuration.

When it runs / caller. Used by npm/Vite/ESLint rather than Flask.

Calls / dependencies. See statement above; UI components generally call upward callbacks, while Dashboard calls api.js.

Imports. Vite React/Tailwind plugins and ESLint config.

Important logic. package.json scripts are dev, build, lint, preview.

Input -> output. Props, browser events, or route -> Rendered UI or API action

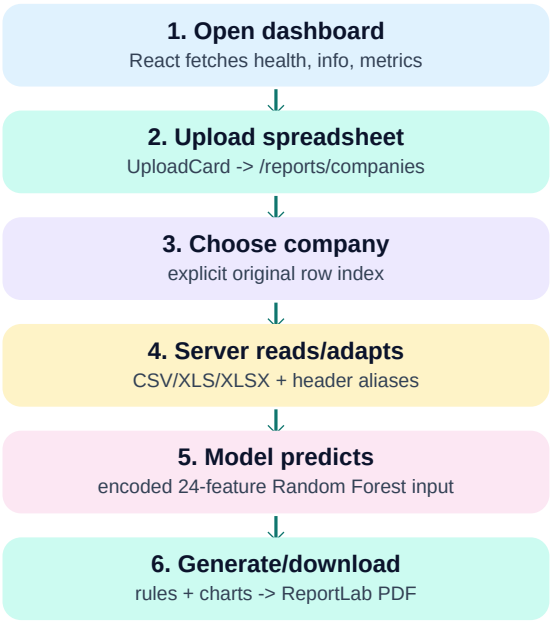
Error handling. API errors are shown in Dashboard toast; rendering uses optional chaining/empty states.

30-second viva explanation

package.json, vite.config.js, eslint.config.js, index.html is part of the React presentation layer. It keeps the interface modular while Dashboard and api.js coordinate data and backend communication.

6. Project Workflow

Selected-company PDF workflow



Step-by-step in plain English

1. The Vite browser app starts and Dashboard calls /health, /model-info, and /metrics in parallel.

2. The user uploads a spreadsheet. The browser sends the file as multipart FormData to /reports/companies.

3. Flask reads CSV with pandas or Excel with pandas/openpyxl, removes fully empty rows/columns, and finds a company-name column through aliases.

4. The browser shows company labels such as Company Name (Row 4). The user must select one; this prevents an accidental report for row one.

5. The selected row and original upload are posted to /reports/generate.

6. The backend maps input headers to model features, fills unavailable categorical features with Not Available and numeric features with 0, then runs prediction.

7. The report module reads original company fields for business context, not the encoded values. It creates only charts/claims supported by uploaded values.

8. Flask returns a PDF attachment. React turns it into a Blob URL for iframe preview and download.

Single prediction workflow

The form is pre-filled from /model-info sample_input. Clicking Analyze startup sends /predict. The response is rendered by PredictionCard and RiskGauge. This path does not make a PDF.

7. API Documentation

Endpoint	Purpose	Request	Response
GET /	API discovery.	None	Service/status/endpoint list.
GET /health	Check server and artifacts.	None	healthy/unhealthy, model_loaded, backend_running; 503 if unavailable.
GET /model-info	Supply UI schema/metadata.	None	Algorithm, feature names/count, sample input, feature importance, sample count.
GET /metrics	Return held-out metrics.	None	accuracy, precision, recall, f1, roc_auc.
POST /predict	Score one startup.	JSON {data: {features}} or direct record.	prediction, probability, confidence, risk_level.
POST /predict/csv	Score all upload rows.	FormData file (.csv/.xlsx/.xls).	rows_predicted, results array. API exists but current UI does not wire it.
POST /reports/companies	List selectable company rows.	FormData file.	companies [{row_index,name,label}], rows_available.
POST /reports/generate	Create selected report.	FormData file and row_index.	application/pdf attachment.

Request example

```
{"data": {"Company": "Example AI", "YCYear": 2024, "... all returned features ...": "..."}}
```

API contract rule

For direct prediction, send all features returned by /model-info. The browser does this by starting from a server-provided complete sample record.

8. Frontend Guide

React uses local state in Dashboard.jsx rather than Redux or Context. `data` holds API startup data, `form` stores editable model input, `prediction` holds the latest result, `companies/selectedRow` handle the report selection, and `reportUrl` is an in-memory browser Blob URL.

User interaction map

User action	Component	API / state effect
Open app	Dashboard	getDashboardData() concurrently requests health, model info, metrics.
Edit fields and analyze	Dashboard + PredictionCard	predictStartup(form) -> /predict -> setPrediction.
Upload file	UploadCard	getReportCompanies(file) -> companies state.
Choose company	UploadCard	onCompanyChange sets selectedRow.
Generate PDF	Dashboard	generateDueDiligenceReport(file,row) -> Blob URL -> iframe.
Download PDF	Dashboard	Temporary anchor uses Blob URL/file name.

9. Backend Guide

Flask route functions remain deliberately thin. `app.py` validates HTTP-level requirements and delegates data/ML/report work. This separation makes prediction reusable outside HTTP and keeps PDF logic out of route functions.

Module	Business logic	Validation / errors
app.py	Orchestration and response format.	BadRequest/PredictionError/413 JSON responses.
predict.py	Artifact-compatible inference.	Artifact presence/type/interface and prediction failures.
dataset_preprocessing.py	Upload reading/header adaptation/row selection.	Empty files, company column, selected row.
insight_generator.py	Deterministic recommendation/risk content.	Treats missing metrics as disclosure gaps.
pdf_generator.py	Document composition.	Requires exactly one record; optional data displays safely.

10. AI Module

The model is `RandomForestClassifier(n_estimators=300, max_depth=15, random_state=42)`. Training uses 24 features and a target called Fate. Categorical columns are LabelEncoded, including the target. The decoder mapping saved in this repository is Dead=0, Exited=1, Operating=2.

Decision process

- A prediction request is aligned to features.pkl so column order matches training.
- Saved encoders transform categorical values. Known categories use the learned mapping; unseen values use 0 and log a warning.
- The forest outputs a predicted class and class probabilities. The maximum probability becomes Probability; multiplied by 100 it becomes Confidence.
- app.py maps outcome label to a presentational risk label. The PDF makes a separate rule-based recommendation from risk, confidence, funding, revenue, growth, and runway where supplied.

Prompt flow

None exists. Any answer claiming the report sends prompts to OpenAI would be inaccurate for this repository. The report is deterministic Python code.

Limitations

- Raw class distribution is imbalanced: Operating 452, Dead 85, Exited 80.
- Saved held-out metrics show 77.4% accuracy and 0.887 weighted ROC-AUC, but the saved report has zero recall for Exited; accuracy alone is not enough.
- Text identifiers such as company name, profile URL, and logo encode into arbitrary numbers and may not generalize.
- Fallback encoding and zero-filled missing numeric inputs can reduce trustworthiness. Human diligence is required.

11. Report Generation

`pdf_generator.generate_report()` receives the unencoded selected row, prediction dictionary, saved metrics dictionary, destination path, and source filename. It uses ReportLab flowables (Paragraph, Table, PageBreak, Image) and a page template for headers/footers.

Report area	Data source	Meaning
Executive summary	report_context()	Model confidence plus evidence/disclosure wording.
Company overview	Original upload via semantic mapping	Industry, funding, valuation, team, revenue, city, year where available.
Feature importance	model/feature_importance.csv	Top business-relevant model inputs; certain profile/identity-like names are filtered here.
Financial analysis	Original numeric upload values	Matplotlib bars for available revenue/funding/growth/etc.
Risk/SWOT/appendix	Deterministic rules + column mapping	Risk actions, missing data, completeness, method information.

12. Data Flow Between Files

Training: CleanStartups.csv -> preprocess.py -> processed_startups.csv and label_encoders.pkl -> train_model.py -> model artifacts. Runtime: Dashboard/api.js -> app.py -> dataset_preprocessing.py/predict.py -> prediction -> pdf_generator.py -> output/pdf -> browser.

13. Libraries Used

Library	Purpose	Primary files
Flask / Flask-CORS	REST API and CORS.	app.py
pandas	Table IO, transformations, numeric parsing.	Most Python modules
scikit-learn	Encoders, Random Forest, metrics/split.	preprocess.py, train_model.py, predict.py
joblib	Artifact persistence.	preprocess.py, train_model.py, predict.py
ReportLab	PDF layout.	pdf_generator.py
Matplotlib	Chart PNGs.	chart_generator.py
Axios	HTTP client/error handling.	api.js
Recharts	Browser charts.	Charts.jsx, FeatureImportance.jsx
Framer Motion	Animations.	Hero/About/PredictionCard
Lucide React	Icons.	Frontend components

14. Common Errors and Debugging

Symptom	Likely cause	Debug / resolution
/health is unhealthy	Artifacts absent/corrupt.	Run python train_model.py; read Flask log for PredictionError.
Connection toast	Flask down or wrong base URL.	Start app.py, check port 5000, set VITE_API_BASE_URL.
Company column error	No mapped company header.	Use Company Name/Company/Startup Name or extend ALIASES.
Report lacks metrics	Upload has no usable numeric fields.	Include revenue/funding/burn/runway/growth; system intentionally avoids guessing.
Excel read failure	Bad workbook/dependency.	Confirm file and openpyxl installation.
Unexpected prediction	Data/model limitations or fallback category.	Inspect logs, input columns, feature order, encoders, class metrics.

15. Future Scope

- Add automated tests for routes, data mapping, prediction schemas, and PDF content.
- Improve minority-class results with additional/cleaner data, class weighting/resampling, tuning, and per-class acceptance criteria.
- Remove or redesign identifier-like features such as URLs/logos/company names; add meaningful financial/market features.
- Add model/dataset versioning, training metadata, experiment tracking, calibrated confidence, and explainability checks.
- Secure production deployment: authenticated users, restricted CORS, debug off, rate limits, virus/file scanning, database/object storage, audit trails.
- Wire the existing batch prediction endpoint to Dashboard and the batch distribution chart.
- Add human-review workflow and explicit policy disclaimers.

16. Team Handover Guide

First files to read

1) app.py for public contract. 2) predict.py for inference. 3) preprocess.py and train_model.py for the model lifecycle. 4) dataset_preprocessing.py/column_mapping.py/insight_generator.py/pdf_generator.py for reports. 5) api.js then Dashboard.jsx for browser coordination.

Adding a feature

First locate its layer. A new uploaded metric may require alias mapping, report extraction, PDF presentation, and tests. A new model feature requires data preparation, retraining, regenerated artifacts, backend schema confirmation, and frontend form confirmation. Avoid manually changing pickled files.

Debug sequence

Check /health -> /model-info -> /metrics before the UI. Reproduce one request outside React if necessary. Then inspect Flask logs, browser network responses, selected input columns, and generated output/pdf files.

17. Frequently Asked Viva Questions

The answers below are based on the implementation, including its limitations. They are intentionally direct enough to rehearse.

1. What type of project is this?

A full-stack machine-learning application: React frontend, Flask REST backend, scikit-learn classifier, spreadsheet processing, and ReportLab PDF generation.

2. What is the model target?

The Fate column. Its saved label mapping is Dead=0, Exited=1, Operating=2.

3. Does it use OpenAI?

No. The repository has no OpenAI dependency or API call. AI here is a Random Forest model.

4. Why use Random Forest?

It handles nonlinear relationships, multiclass output, mixed encoded inputs, and gives feature importances/probabilities with a practical training workflow.

5. How many trees are trained?

300 trees, with maximum depth 15 and random_state 42.

6. Why set random_state?

It makes split and model behaviour reproducible across runs, as far as the library permits.

7. How is the dataset split?

train_test_split uses 20% test data, 80% training data, and stratify=target.

8. Why stratify?

The outcome classes are imbalanced, so stratification preserves their proportions in train and test sets.

9. What is LabelEncoder doing?

It maps each categorical string to an integer. A separate fitted encoder is saved for each categorical column.

10. Why save encoders?

Prediction must apply the same category-to-number mappings that training used.

11. What happens for unseen categories?

Inference logs a warning and uses fallback encoding 0. It keeps the API available but is a modelling limitation.

12. Why is features.pkl needed?

It preserves the exact expected feature names and order for classifier inference.

13. What does confidence mean?

It is the maximum class probability from predict_proba multiplied by 100, not a guarantee of real-world success.

14. How does risk level work?

app.py maps Operating to Low, Exited or Acquired to Medium, and all other labels to High.

15. Is risk a second trained model?

No. It is a simple presentation rule over the outcome label.

16. How is report recommendation decided?

Deterministic conditions use risk, confidence, growth, runway, funding, and revenue to choose Strong Buy, Buy, Hold, Watchlist, or Avoid.

17. Does the report invent missing facts?

No. It detects missing metrics and writes disclosure-gap/verification statements instead.

18. Why keep original upload data for reports?

Encoded values are only meaningful to the model. A human report needs original company names and actual business values.

19. Why require a selected row?

To prevent silently reporting the first spreadsheet row when the user intended another company.

20. Which uploads are allowed?

CSV, XLSX, and XLS, subject to a 10 MB Flask upload limit.

21. How are variable headers handled?

Aliases and normalized exact/partial matching in column_mapping.py map common terms to canonical concepts.

22. What does Flask do?

It receives HTTP requests, validates them, calls local modules, and returns JSON or a PDF file.

23. Why use CORS?

During development Vite and Flask run on different origins/ports, so the browser needs permission for API calls.

24. What is the use of /health?

It verifies both backend availability and that model artifacts can load, returning 503 if they cannot.

25. What is returned by /model-info?

Model algorithm/name, features, feature count, sample input, importance records, target, and training sample count.

26. How does the dashboard avoid missing fields?

It asks /model-info for a sample input and edits that complete record rather than starting with an unknown schema.

27. What does Axios add?

A central base URL, timeout, request helpers, multipart handling, and consistent error-message extraction.

28. How does PDF preview work?

The response is a blob; Dashboard creates a browser object URL and uses it as an iframe source.

29. Why revoke Blob URLs?

To reduce browser memory leakage when a new report replaces an old report or the component unmounts.

30. Which library makes the PDF?

ReportLab. Matplotlib separately makes chart PNG files embedded in the document.

31. What chart data is used?

Available selected-row values for revenue, funding, employees, growth, burn rate, runway, or valuation.

32. How are model metrics computed?

On held-out test data: accuracy, weighted precision/recall/F1, ROC-AUC, confusion matrix, classification report.

33. Why weighted metrics?

They account for each class while weighting contributions by support, which matters with imbalanced class counts.

34. What limitation does the current report show?

Saved metrics show zero recall for the encoded Exited class, despite 77.4% overall accuracy; minority-class performance needs work.

35. Why is accuracy insufficient?

The Operating class dominates, so a model may look accurate while missing smaller but important outcomes.

36. What is check_data.py for?

A manual utility that prints processed Fate counts and dtypes; it is not used by the web application.

37. What is frontend/dist?

The generated production build from Vite. It should be rebuilt, not edited directly.

38. Is Tailwind actively responsible for all styling?

It is installed/configured, but the visible project styles are primarily custom CSS in index.css and App.css.

39. Why is Sidebar not visible?

Sidebar.jsx renders with inline display:none, so it is currently a placeholder.

40. Is batch prediction complete in the UI?

The API and api.js function exist, but Dashboard does not call predictCsv or pass batchResult to Charts, so the batch distribution is not active.

41. What error type represents domain failures?

PredictionError, a ValueError subclass used for artifacts, uploads, and prediction problems.

42. How does Flask return invalid requests?

BadRequest is handled as JSON 400, not a default HTML error page.

43. How are missing numeric values handled in training?

They are filled with the column median, or zero if the entire numeric column is missing.

44. How are missing categorical values handled in training?

They are filled with the mode, or Unknown if there is no mode.

45. How does report adaptation fill absent features?

Categorical model features receive Not Available; numeric features receive 0, allowing an imperfect but complete inference record.

46. What security improvements are needed?

Restrict CORS, disable debug in production, authenticate users, scan uploads, limit/rate-limit requests, and store reports safely.

47. How would you improve model validity?

Audit leakage/identifier features, collect better balanced data, test models, tune/calibrate, and set per-class performance goals.

48. What happens after raw data changes?

Run train_model.py to regenerate processed data, encoders, model, metrics, features, and importance before serving predictions.

49. What must a new developer read first?

app.py for contract, predict.py for inference, training/preprocessing for lifecycle, then api.js/Dashboard for frontend orchestration.

50. What is the most important ethical warning?

The tool is decision support. It should not replace verification or professional investment, legal, financial, security, or tax due diligence.

18. Quick Revision

One-minute summary

VentureSignal is a React + Flask project that trains a 300-tree Random Forest on historical startup records. Flask loads saved model artifacts to predict Dead, Exited, or Operating. Users can submit a single form or upload a spreadsheet and select a company for a deterministic ReportLab PDF.

Five-page revision checklist

Page A - Architecture

React calls Flask via Axios. Flask calls prediction/report modules. Training creates artifacts under model/.

Page B - ML

preprocess.py cleans/fills/encodes. train_model.py stratifies 80/20, trains 300-tree RF, calculates metrics, saves artifacts.

Page C - Prediction

predict.py loads artifacts, aligns feature order, encodes text, predicts class/probability, decodes Fate. app.py maps risk.

Page D - Report

Upload -> company list -> selected row -> header mapping -> prediction -> deterministic insights/charts -> ReportLab PDF -> browser Blob preview.

Page E - Caution

No OpenAI/LLM. Imbalanced classes and identifier-like encoded features are limitations. Report is decision support and requires human verification.

Final team checklist

- Run training before first backend start or after dataset changes.
- Verify /health before debugging the frontend.
- Keep frontend API base URL aligned with Flask port.
- Never hand-edit model pickle files or frontend dist bundles.
- Update README and this training manual when endpoint/data-flow behaviour changes.

End of manual

This training manual reflects the repository as inspected on 07 August 2026. Use it alongside the code and test data when making production decisions.